

701968234901768023405879312016
670980321768767

WVPOYFXI LFJXI G I E
RST MPHLOM ZFOLI PG
BXINYSXWHYGI JLGIL
OLI EPCOOOTJPLQDZC
NYDWIGLUPJLQDZC

Week 12

2. Sockets (I) – Introducing Sockets

Dr. Andreas S. Maniatis

Assistant Professor

School of Computer Science

COMP.2560 – System Programming [2025F]

Monday, November 19th, 2025



University of Windsor

Content

- Networking (Review)
- Introducing Sockets
- Creating endpoints for communication: `socket()`
- Socket Address Structures:
 - Internet Socket Address structure
 - Unix Socket Address structure
 - Generic Socket Address Structure
- Initiating a connection on a socket: `connect()`
- Binding a name to a socket: `bind()`
- Listening for connections on a socket: `listen()`
- Accepting a connection on a socket: `accept()`

Part I

Networking (Review)



Network models

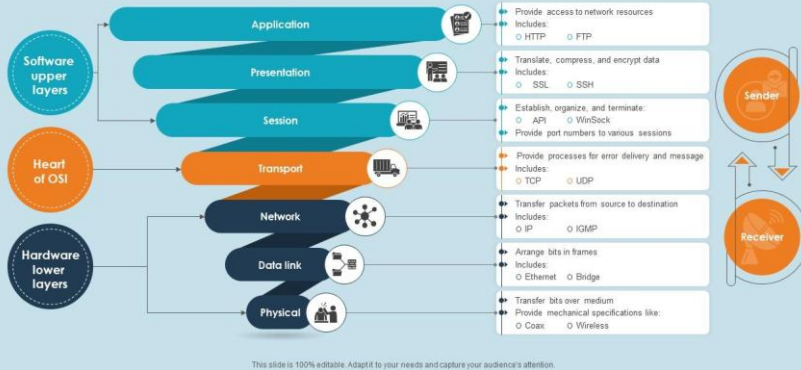
Computer Network Model describes how information is transmitted between two points in a network. Two models:

- The **OSI model** (Open Systems Interconnect) proposed by ISO describes network activities as having a structure of **seven layers**, each of the layers has one or more protocols associated with it. → Theoretically sound.
- The **TCP/IP model** (Transmission Control Protocol/Internet Protocol) based on the TCP/IP protocol suite. → **More practical.**

The OSI Reference Model

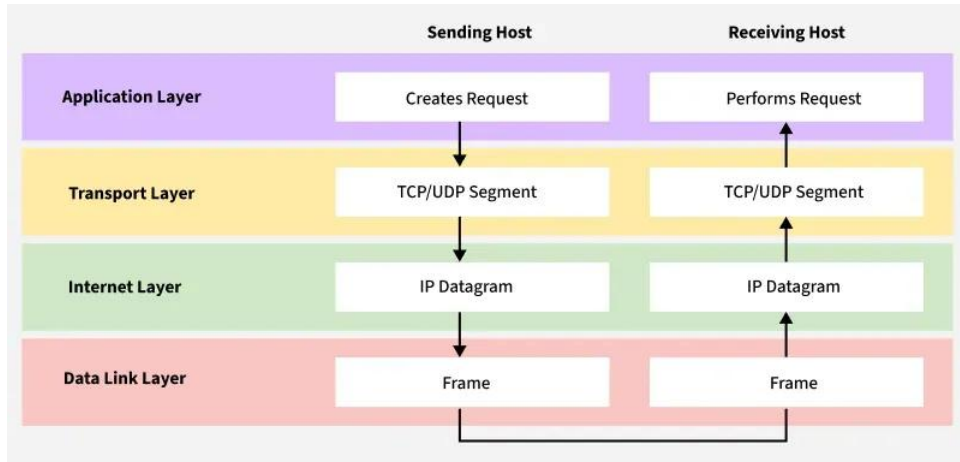
7 layers of OSI model to enhance computer system communication

This slide showcases layers of OSI model that help organizations understand various networking protocol interactions in networked environment. It includes layers such as physical layer, data link, network, transport, session, etc.

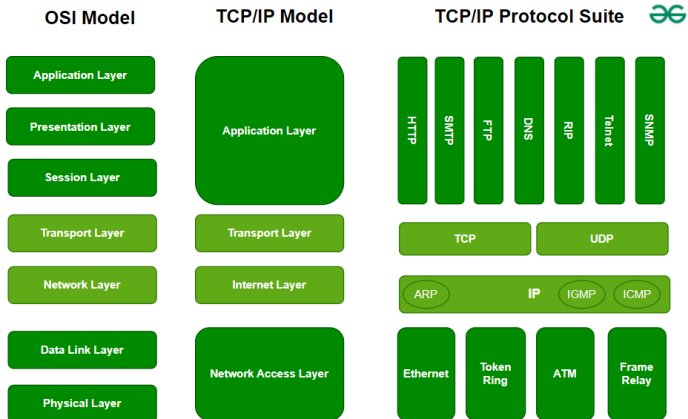


This slide is 100% editable. Adapt it to your needs and capture your audience's attention.

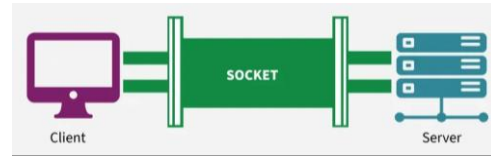
The TCP/IP Model



TCP/IP vs OSI



IP-Address:Port → Socket



- **IP Address:** A unique numerical label assigned to devices connected to a computer network for communication, like a mailing address. Logical constructs that identify a specific process or network service on a device.
- **Port:**
 - Usually a 16-bit number ranging from 1 to 65,535.
 - Helps sort incoming traffic to the correct application, preventing a single device from being overwhelmed by multiple services.
- **How they work together:**
 - An IP address and a port number together form a complete communication endpoint, sometimes called a **socket**.
 - The IP address gets the data to the right computer, and the port number directs it to the correct application on that computer.

Part II

Introduction to Sockets



Creating endpoints for communication: `socket()`

Socket Address Structures

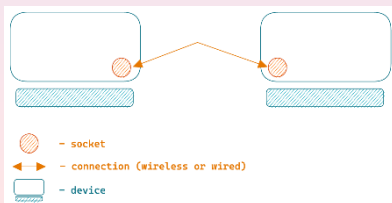
Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets I

- Sockets are the traditional UNIX IPC mechanism that allows **local/distant** processes to talk to each other.
- IPC using sockets is based on the **client/server** paradigm.

Note

Socket connections are **bidirectional**.



Creating endpoints for communication: `socket()`

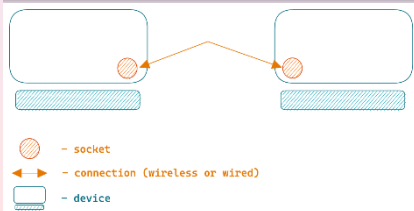
Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets I

The first widespread implementation of the **sockets API** appeared with 4.2BSD in **1983**, and this API has been ported to virtually every UNIX implementation, as well as most other operating systems.

- Unix domain sockets (Datagram sockets)
- Internet domain sockets (Stream sockets)



Introducing Sockets

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`

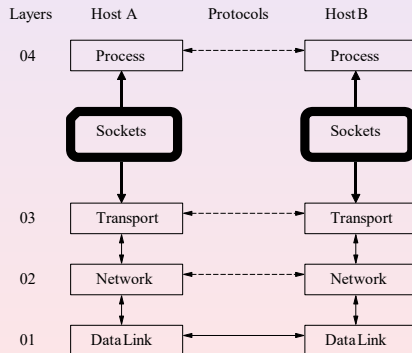
Binding a name to a socket: `bind()`

Listening for connections on a socket: `listen()`

Accepting a connection on a socket: `accept()`

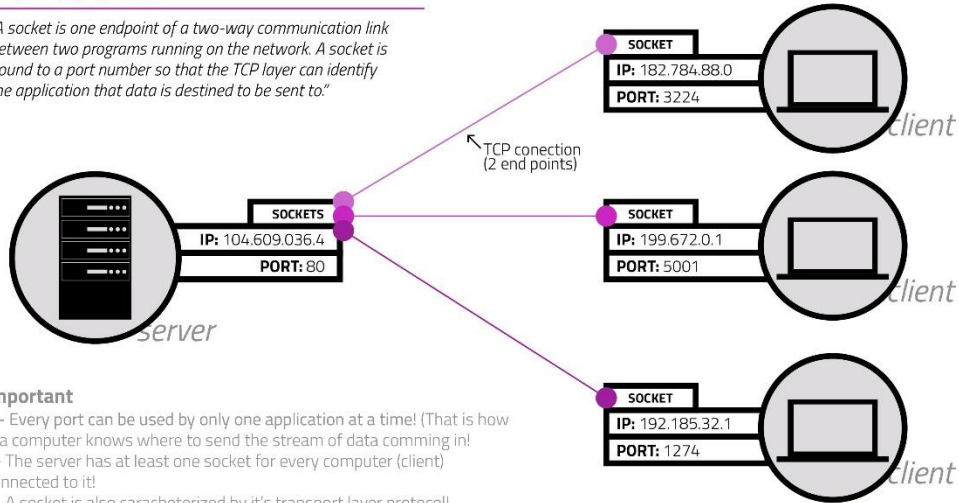
Although possible, it is difficult and time consuming to use directly the TCP/IP protocol to implement network applications.

⇒ The socket interface acts between the user process and TCP/IP



TCP SOCKET

"A socket is one endpoint of a two-way communication link between two programs running on the network. A socket is bound to a port number so that the TCP layer can identify the application that data is destined to be sent to."



Important

- 1 - Every port can be used by only one application at a time! (That is how the computer knows where to send the stream of data coming in)
- 2- The server has at least one socket for every computer (client) connected to it!
- 3- A socket is also characterized by its transport layer protocol! (TCP, UDP, etc)

○ Sockets: Typical scenario



Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario (stream sockets)

In a typical client-server scenario, applications communicate using sockets as follows:

- Each application **creates** a socket. A socket is the “apparatus” that allows communication, and both applications require one.
- The server **binds** its socket to a well-known address (name) so that clients can locate it.

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario

A socket is created using the `socket()` system call, which returns a socket **file descriptor** used to refer to the socket in subsequent system calls:

```
fd = socket(domain, type, protocol);
```

More details on this call later.

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario

The typical scenario can be described in a bit more details as follows:

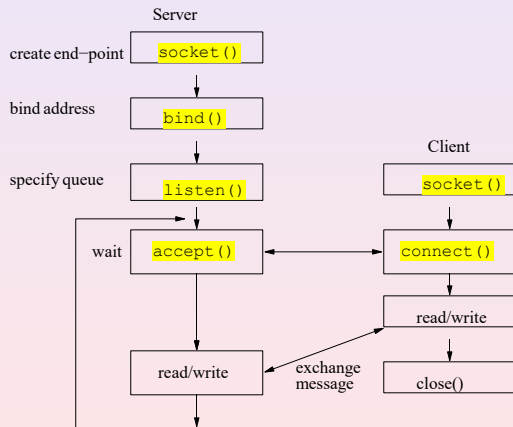
- The **server process** creates **socket** whose name is known by the client processes and **listens** on that socket for requests from clients.
- A **client process** can talk to the server process by:
 - creating a **socket** and then,
 - **requesting** it to be connected to the server's named socket
- If successful, one **file descriptor** is returned to the client and **another one** to the server. These file descriptors can be used for **read** and **write**, allowing the server and client to communicate.

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario



Creating endpoints for communication: `socket()`

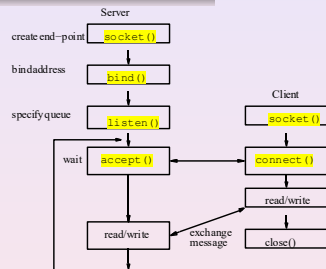
Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario

The key socket system calls are the following:

- The `socket()` system call creates a new socket.
- The `bind()` system call binds a socket to an address. Usually, a server employs this call to bind its socket to a well-known address so that clients can locate the socket.
- The `listen()` system call allows a stream socket to accept incoming connections from other sockets.
- The `accept()` system call accepts a connection from a peer application on a listening stream socket and, optionally, returns the address of the peer socket.
- The `connect()` system call establishes a connection with another socket.



Creating endpoints for communication: `socket()`

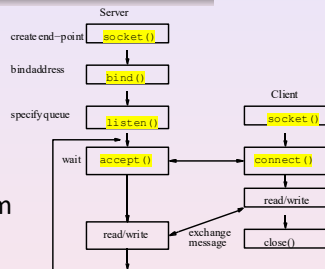
Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario

The operation of stream sockets can be explained by analogy with the telephone system:

1. The **`socket()` system call**, which creates a socket, is the **equivalent of** installing a telephone.
In order for two applications to communicate, each of them must create a socket.



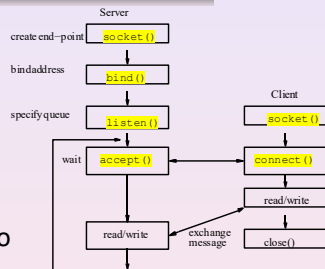
Creating endpoints for communication: `socket()`

Socket Address Structures

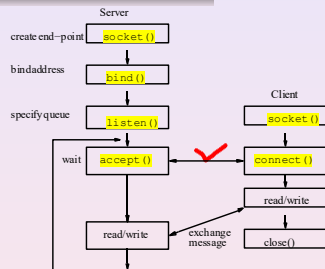
Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario

2. Communication via a stream socket is analogous to a telephone call. One application must connect its socket to another application's socket before communication can take place. Two sockets are connected as follows:
 - a) One application calls `bind()` in order to bind the socket to a well-known address and then calls `listen()` to notify the kernel of its willingness to accept incoming connections. This step is **analogous to** having a known telephone number and ensuring that our telephone is turned on so that people can call us.



- b. The other application establishes the connection by calling **connect ()**, specifying the address of the socket to which the connection is to be made. This is **analogous to** dialing someone's telephone number.
- c. The application that called **listen ()** then accepts the connection using **accept ()**. This is **analogous to** picking up the telephone when it rings. If **accept ()** is performed before the peer application calls **connect ()**, then **accept ()** blocks ("waiting by the telephone").



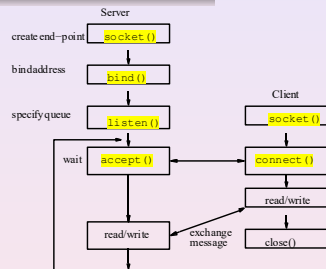
Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Sockets: Typical scenario

- Once a connection has been established, data can be transmitted in both directions between the applications (**analogous to** a two-way telephone conversation) until one of them closes the connection using **close()**. Communication is performed using the conventional **read()** and **write()** system calls or via a number of socket specific system calls (such as **send()** and **recv()**) that provide additional functionality.



Part II

Creating Endpoints



Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

The `socket()` system call

`socket()`

Synopsis:

```
int socket(int domain, int type, int protocol);
```

`socket()` creates an endpoint for communication and returns a **file descriptor** referencing the socket.
In case of failure, `socket()` returns -1.

Note

Calling `socket()` is **the first thing** a process must do in order to perform any network I/O operation.

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Socket's attributes

Three attributes may differentiate between different kinds of sockets:

1. **The domain:** `AF_INET` for internet and `AF_UNIX` for same machine IPC.
(Note that `AF` stands for Address Family).
2. **The type of communication:** `SOCK_STREAM`, reliable byte stream connection (TCP) and, `SOCK_DGRAM`, unreliable connectionless (UDP).
3. **The protocol:** The low-level protocol used for communication. This parameter is usually set to `0` in system calls, which means “use the correct/default protocol”.

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Socket's attributes

Domain	Communication performed	Communication between applications	Address format	Address structure
AF_UNIX	within kernel	on same host	pathname	<i>sockaddr_un</i>
AF_INET	via IPv4	on hosts connected via an IPv4 network	32-bit IPv4 address + 16-bit port number	<i>sockaddr_in</i>
AF_INET6	via IPv6	on hosts connected via an IPv6 network	128-bit IPv6 address + 16-bit port number	<i>sockaddr_in6</i>

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Socket's attributes

Every sockets implementation provides at least two types of sockets:

- **Stream** and
- **Datagram.**

These socket types are supported in both the UNIX and the Internet domains.

Property	Socket type	
	Stream	Datagram
Reliable delivery?	Y	N
Message boundaries preserved?	N	Y
Connection-oriented?	Y	N

Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

Example

When a reliable byte-stream connection is requested across the internet:

```
sd = socket(AF_INET, SOCK_STREAM, 0);
```

`man socket``man 7 socket`

The `bind()` system call binds a socket to an **address**.

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

Typically, we bind a server's socket to a well-known address—that is, a fixed address that is known in advance to client applications that need to communicate with that server.

First, let us talk about Address Structure.

Part III

Socket Address Structure



Socket Address Structure

Most socket functions require an argument pointer to a socket address structure, **which is protocol dependent**.

In particular, we have:

1. **Internet socket** address structure,
2. **Unix socket** address structure,
3. **Generic Socket** Address Structure

Socket Address Structure

```
struct type1 {  
    int i;  
    int k;  
} ;
```

```
struct type2 {  
    char arr1[2];  
    char ar2[6]  
} ;
```

```
struct generic_type  
{  
    int i[2];  
} ;
```


Socket Address Structure

```
struct type1 {  
    int i;  
    int k;  
};
```

```
struct type2 {  
    char arr1[2];  
    char arr2[6];  
};
```

```
struct generic_type  
{  
    int i[2];  
};
```

```
void fun(struct generic_type  
*p);
```

```
Struct type1 tp1;  
Struct type2 tp2;
```

```
fun( (struct generic_type *)  
&tp1);  
fun( (struct generic_type *)  
&tp2);
```

Internet socket address structure

Internet socket address structure

The structure is called `sockaddr_in`, defined in `<netinet/in.h>` ?

<http://man7.org/linux/man-pages/man7/ip.7.html>

man 7 ip

Here is a version for **IPv4**:

```
typedef uint32_t in_addr_t;
typedef unsigned short sa_family_t;

struct in_addr {
    uint32_t s_addr;
};

struct sockaddr_in {
    sa_family_t    sin_family;    // AF_INET
    in_port_t      sin_port;      // 16-bit port number
    struct in_addr sin_addr;      // 32-bit IPv4 address
    char           sin_zero[8];   // unused
};
```

Unix socket address structure

The structure is called `sockaddr_un`, defined in `<sys/un.h>`

Here is a version from **Solaris**:

```
struct  sockaddr_un {  
    sa_family_t  sun_family;      // AF_UNIX  
    char         sun_path[108];   // Path name  
};
```

man 7 unix

<https://linux.die.net/man/7/unix>

Generic Socket Address Structure

- *Socket address structures* are always passed by address when passed as a parameter.
- Because there are several kinds of socket structures, **socket functions prototypes take a pointer to the generic socket address structure**, which represents **any** socket address structure parameter.

The generic address structure is called `sockaddr`, defined in `<sys/socket.h>`

Here is the definition of the structure:

```
struct  sockaddr{                                     man 2 bind
    uint8_t      sa_len;
    sa_family_t  sin_family;
    char         sa_data[14]; // protocol-specific address
};
```

Part IV

Socket use flow



○ Sockets: `bind()`



Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

The `bind()` system call

`bind()`

Synopsis:

```
int bind(int s, struct sockaddr *sp, int len);
```

Returns 0 when successful and -1 otherwise.

`bind()` assigns a local protocol address to a socket.

In case of the Internet, the protocol address consists of a 32-bit IPv4 address and a 16-bit port number.

`bind()` is usually called by a server to bind their local IP and a well-known port number to a socket.

Example

```
struct sockaddr_in sv; // IPv4 socket
...
sd = socket(AF_INET, SOCK_STREAM, 0);
...
bind(sd, (struct sockaddr*)&sv, sizeof(sv));
```

Example

Example

Example : The `bind()` function prototype is

```
int bind(int, struct sockaddr *, socklen_t);
```

→ Any call to this function should **cast** the pointer to the protocol-specific socket address structure to be a point to a generic socket address structure.

For example:

```
struct sockaddr_in sv; // IPv4 socket, server
:
bind(sfd, (struct sockaddr*)&sv, sizeof(sv));
```


○ Sockets: `listen()`



Creating endpoints for communication: `socket()`

Socket Address Structures

Initiating a connection on a socket: `connect()`Binding a name to a socket: `bind()`Listening for connections on a socket: `listen()`Accepting a connection on a socket: `accept()`

The `listen()` system call

`listen()`

Synopsis: `int listen(int s, int backlog);`

Returns 0 when successful and -1 otherwise.

- `listen()` is called only by a TCP **server** to accept connections from client sockets that will issue a `connect()`.
- `s` is a file descriptor of a socket that has been already created.
- `backlog` defines the maximum length the queue of pending connections may grow to.
- `listen()` is normally called after the calls to `socket()` and `bind()`.

See notes in `man listen`

○ Sockets: connect ()



The `connect()` system call

`connect()`

Synopsis:

```
int connect(int s, struct sockaddr *srv, int len)
```

Returns 0 when successful and -1 otherwise.

`connect()` is used by a TCP **client** to establish a connection with a TCP server.

man connect

The parameters have the following meanings:

- `s` is a socket descriptor that was returned by `socket()`.
 - `srv` is a pointer to a socket address structure object, which must contain the IP address and port number **of the server**.
- `len` is the size of the socket address structure.

`connect()` only returns when a connection is established or when an error occurs.

○ Sockets: accept ()



The `accept()` system call

`accept()`

Synopsis:

```
int accept(int s, struct sockaddr *addr,  
socklen_t *addrlen);
```

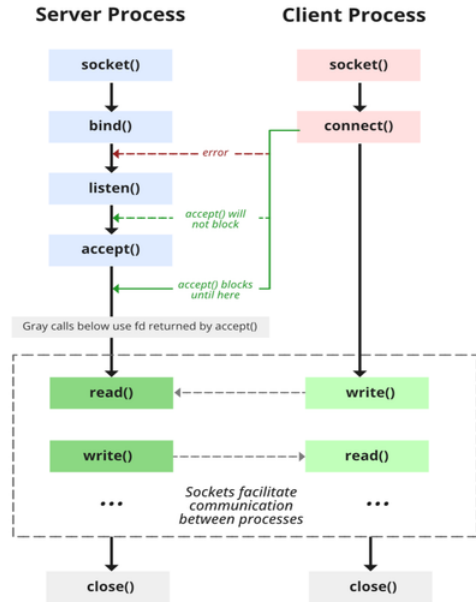
See `man accept`

Returns a file descriptor for a new socket when successful and -1 otherwise.

- `accept()` is called by a TCP server to extract the first connection on the queue of pending connections, creates a new socket with the properties of `s`, and allocates **a new file descriptor** for the newly created socket.
- If no pending connections are present on the queue, `accept()` **blocks the caller** until a connection is present. Usually, the file descriptor `s` is called **the listening socket**, while the returned value is called the **connected socket**.

Socket Flow: Review

- Server:
 - Create Socket
 - `setsockopt`
 - Bind
 - Listen
 - Accept
 - Send/Receive
 - Close
- Client:
 - Create Socket
 - Connect
 - Send/Receive
 - Close
- Common Issues:
 - Connection failure
 - Port binding errors (already in use)
 - Blocking sockets (default)



Part IV

Attendance Quiz (Not graded)



Quiz



Attendance Quiz

701968234901768023405879312016
670980321768767



774862586034187069210783468019
701968234901768023405879312016
670980321768767

WPOYFXI LFJXI G I E
H I MPHLOM ZFOLI PG
BXNYSXWHYGI JLG I L
OLI EPC OOUJPL Q DZC
NYDWIGL OUPJL Q DZC

Thank you!
Questions?

DEFYXHQGRUOSGRP RN	3236343972474116
QJRYVQUXVRI DKUEVP	893833239594578
KGWBGQFHP RSUP RU	0310019701552750
XURNYUXZ DCSYUUM I	7875465400196596
WHMFSP UIQYSUQYRSHRG	5019353606937419

00110011100111
11111010000100
11000000001111
00111100111100
00101111001100
00000111011011
11001100110011

Dr. Andreas S. Maniatis

Assistant Professor

Andreas.Maniatis@UWindsor.ca

<http://www.linkedin.com/in/andreasmaniatis>



University of Windsor